43008: Reinforcement Learning

Assignment -1

Student Name: Hyunbeen Suk

Student ID: 14655765

Selected Scenario:

1. Dynamic Pricing for a Ride-Sharing Platform

2. Energy Management in a Smart Grid

Introduction

This report discusses two cases of practical control problems defined as finite horizon MDPs and solved by using two traditional dynamic-programming (DP) algorithms which are value and policy iterations. Case Study 1 is the required case which is about Dynamic Pricing of a Ride Sharing Platform. In this case, an operator runs a single zone during a two-hour time frame and chooses at intervals of 10 minutes whether to change prices or supply to ensure the balance of rider demand and availability of drivers. Case Study 2 is taken from the optional cases, which is about Energy Management on a Smart Grid. In this case, an operator decides about distribution, storage, or curtailment of energy per hour within a 24-hour day.

Both problems satisfy the stated structural assumptions. The state of the system is fully observable, making each problem a standard MDP problem rather than a POMDP problem. Every action performed results in stochastic outcomes, and every episode is of a constant and known length — 12 decision periods in Case Study 1, and 24 periods in Case Study 2. Thus, a finite horizon model makes sense: the ride-sharing episode finishes in two hours, and the grid problem is solved daily, with a terminal reward assessing the state of the grid when it is transferred to the next day. As the horizon is finite, the optimal policy is non-stationary, since the optimal action for a particular state depends not only on the number of the remaining periods, but also on the time of the day when the decision is being made.

Both algorithms were implemented from scratch:

- Value iteration is implemented as backward induction. Starting from the terminal value $V_T(s)$, a single backward sweep performs T Bellman-optimality backups, $V_t(s) = \max_a [R_t(s,a) + \Sigma_{s'} P_t(s'|s,a) \cdot V_{t+1}(s')]$. In a finite horizon this is exact after exactly T stage-backups.
- Policy iteration alternates an exact backward policy-evaluation sweep (the same recursion with the action fixed by the current policy) with a greedy policy-improvement pass over every (t, s) pair, stopping when a full pass changes no action.

Given that both techniques lead to the Bellman-optimal value function, the two must produce the same optimal policy and value function, providing a form of verification for the entire process. This report will cover the MDP model as well as the reasoning behind each transition/reward model, followed by the experimentation setup, performance and comparison results, advantages and weaknesses of each technique, and a concluding summary.

Markov Decision Process Formulation

An MDP is the tuple (S, A, P, R) over a horizon T with a terminal value V_T. For a finite horizon the policy is a separate mapping $\pi_t : S \to A$ for each $t = 0, \ldots, T-1$, and the objective is to maximise the expected undiscounted return $E[\Sigma_{t=0}^{T-1} R_t(s_t, a_t) + V_T(s_T)]$. No discount factor is applied ($\gamma = 1$): the episodes are short and of fixed length, and the terminal value carries the "what happens after the episode" information.

Case Study 1 — Dynamic Pricing for a Ride-Sharing Platform

Horizon. One zone, a two-hour window, a decision every 10 minutes gives $T = 12$ decision periods ($t = 0 \ldots 11$); the episode ends at $t = 12$.

States (S = 9)

The state summarises the balance between rider demand and available drivers in the zone. A three-level severity scale is used on each side of balance, plus one structural condition and one crisis condition.

#	State	Meaning
0	BALANCED	Supply matches demand — riders picked up quickly, drivers earning.
1	MILD_UNDERSUPPLY	Demand slightly above supply — wait times up a little.
2	MODERATE_UNDERSUPPLY	Clear shortage — noticeable waits, no cancellations yet.
3	SEVERE_UNDERSUPPLY	Large shortage — long waits, riders starting to cancel.
4	MILD_OVERSUPPLY	A few more drivers than requests — minor idle time.

#	State	Meaning
5	MODERATE_OVERSUPPLY	Clear surplus of drivers — meaningful idle time.
6	SEVERE_OVERSUPPLY	Many idle drivers, few requests — drivers likely to log off.
7	LOW_DRIVER_BASE	Structurally few drivers active (late night / break). Looks like a demand spike from outside, but the cause — and the right fix — is a supply-side one.
8	CRITICAL_BACKLOG	Request queue growing faster than it clears — imminent service collapse.

The three-level scale (9 states, more than the minimum required of 6 states) permits the optimal policy to respond gradationally to the problem, responding lightly to a small problem and firmly to a large one. The reason why LOW_DRIVER_BASE is categorized separately from other states is because the case scenario shows that even though it is like a demand surge, it needs supply fixing; hence, keeping them different helps the optimal policy handle them differently. CRITICAL_BACKLOG is categorized as a failure regime close to absorbing state where getting out of it requires very firm action.

Actions (A = 5)

#	Action	Description
0	HOLD	Leave pricing untouched.
1	RAISE_PRICE	Surge pricing — suppresses demand, slowly attracts drivers.
2	LOWER_PRICE	Discount — boosts demand.
3	DRIVER_INCENTIVE	Pay drivers a bonus to enter or stay. Reliable supply boost at a direct cost.
4	PAUSE_NEW_REQUESTS	Temporarily stop accepting new requests so drivers clear the backlog.

These are exactly the five levers named in the case description.

Rewards and rationale for the reward matrix

The per-period reward is R(s, a) = operating_value(s) + action_cost(s, a).

operating_value(s) is a single figure blending rider satisfaction, driver utilisation and operating efficiency:

State	BAL	MILD-U	MOD-U	SEV-U	MILD-O	MOD-O	SEV-O	LOW-DRV	CRIT
value	+10	+5	−1	−7	+4	−1	−5	−3	−16

Rationale: BALANCED is the ideal operating point, with the highest rating. The penalty applied for under-supply is somewhat higher than for the mirrored over-supply situation, regardless of the intensity (−7 compared to −5 in case of “severe”; +5 compared to +4 in case of “mild”) since rider cancellations cause damage to a two-sided marketplace irreversibly, while the cost of idle drivers does not indicate defection. MILD_UNDERSUPPLY rating is positive because a demand-driven area is one where the platform generates revenue. LOW_DRIVER_BASE condition stands between “moderate” and “severe” under-supply, signifying a real service problem but one that is solvable. CRITICAL_BACKLOG is rated negatively as a qualitatively worse, vicious cycle, rather than as “very severe under-supply.”

action_cost(s,a) is defined using the cost of action which follows an ordering based on the cost of the lever being considered, i.e., HOLD(0), RAISE_PRICE(−1), LOWER_PRICE(−2), DRIVER_INCENTIVE(−4), and PAUSE_NEW_REQUESTS(−6). This is followed by a series of effects which include first a soft effect on the rider goodwill, followed by margin concession, thereafter paying cash to the drivers, and finally foregoing the whole trip. Besides the above cost elements, state dependent costs are added in order to capture the explicit tradeoffs mentioned in the case:

- RAISE_PRICE in an already balanced or over-supplied zone: −4 (turning away riders the zone could have served).
- LOWER_PRICE when drivers are scarce (MOD_U, SEV_U, LOW_DRIVER_BASE, CRIT): −4 (pulling in

demand the zone cannot serve).

- DRIVER_INCENTIVE when drivers are already plentiful: −3 (paying for drivers already there).

The terminal reward is V_T(s) = operating_value(s): the platform is credited for leaving the zone in a good state at the end of the two hours.

Transition probabilities and rationale for the transition matrix

Transitions are time-varying, written P_t(s' | s, a), and are built in two stages.

Base model P_base. For every (state, action) pair a distribution over next states is hand-designed following four rules: (i) the mode of the distribution is the action's intended effect; (ii) HOLD mean-reverts toward BALANCED from the mild states; (iii) severity moves one notch at a time (MILD ↔ MODERATE ↔ SEVERE), rarely skipping a level; (iv) the bad states carry a tail risk to an even worse state — for example, under HOLD, SEVERE_UNDERSUPPLY slips to CRITICAL_BACKLOG with probability about 0.18. That tail is what makes "doing nothing" genuinely dangerous and gives the DP solution something to anticipate. Magnitudes are round numbers in a "primary ≈ 0.4 / secondary ≈ 0.3 / alternative ≈ 0.2 / tail ≤ 0.1" pattern; only the ordering and the rough ratios carry meaning, and every row is renormalised to sum to 1. No historical data is provided, so these are design values, as the assignment permits.

Time variation. The two-hour window runs from an evening demand peak into the late night. This is modelled as four phases that reshape P_base through three daily-rhythm channels — none of them the weather, events, accidents or outages the case description excludes:

Phase	periods	demand_pressure	driver_attrition	price_efficacy
A – peak	t = 0–2	1.20	0.00	1.00
B – easing	t = 3–5	0.95	0.10	1.00
C – quiet	t = 6–8	0.05	0.55	0.78
D – late night	t = 9–11	0.00	1.70	0.40

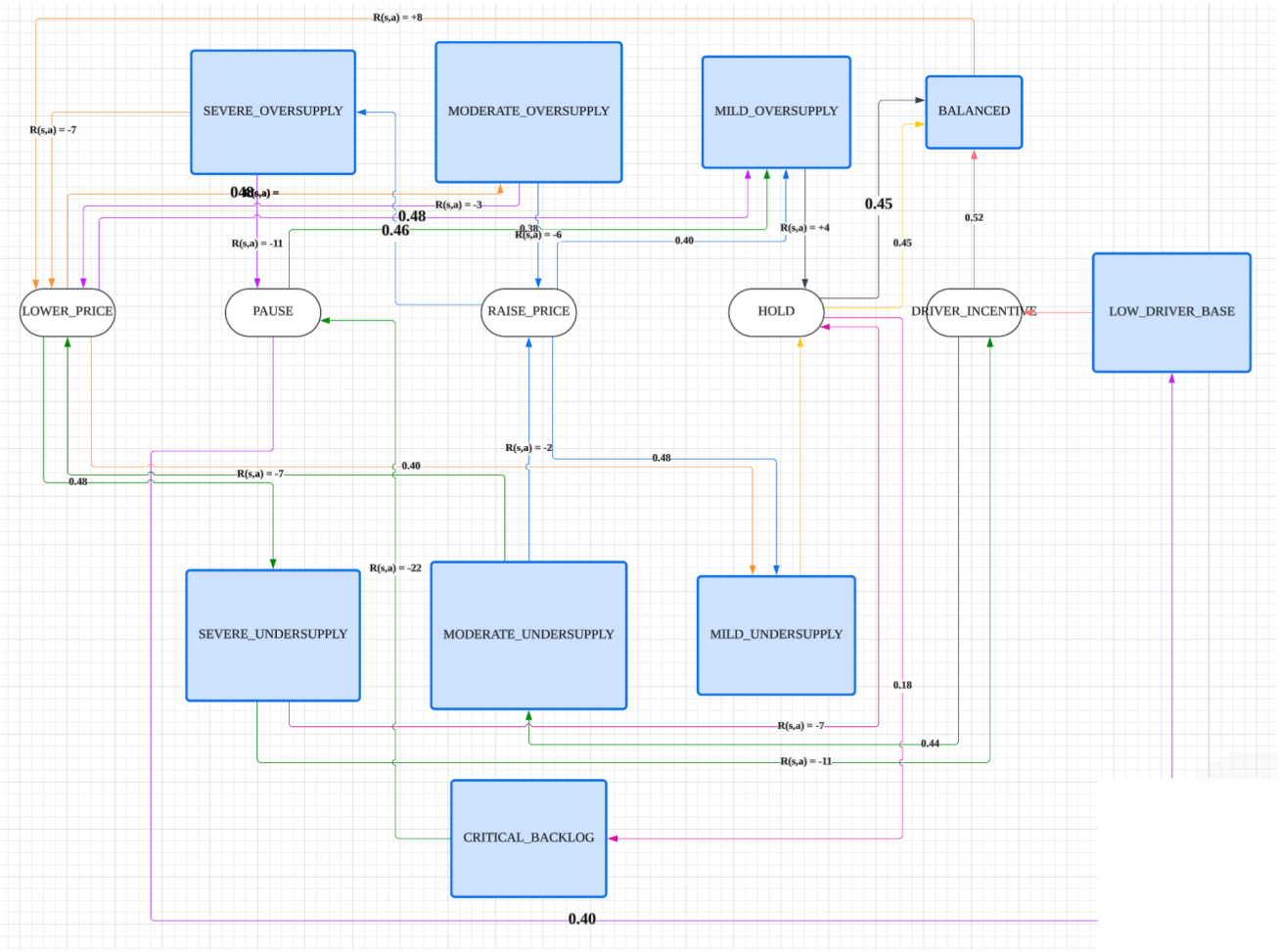
- demand_pressure — net rider inflow relative to driver inflow. It multiplies each next-state probability by exp(0.30 · demand_pressure · scale[a] · lean[s']), where lean[s'] scores how "under-supply-ish" a next state is (−3 for severe over-supply … 0 for balanced … +3 for severe under-supply, +4 for critical). scale[a] is the fraction of this pressure each action is exposed to: the tilt represents unmodelled rider inflow, so a demand-curbing action feels less of it — RAISE_PRICE (0.25) and PAUSE_NEW_REQUESTS (0.35) versus 1.00 for HOLD, LOWER_PRICE and DRIVER_INCENTIVE.
- driver_attrition — the rate at which active drivers log off for the night: low early, high after the peak. It tilts transitions toward LOW_DRIVER_BASE and is action-independent (drivers log off regardless of pricing).
- price_efficacy (≤ 1) — demand's price elasticity. Late at night, few riders are choosing whether to request, so a price change barely moves the tiny request volume; the RAISE_PRICE and LOWER_PRICE transitions are blended toward the HOLD dynamics, while the supply-side levers keep working.

The shape of each channel is fixed by this physical story (demand falls, attrition rises, elasticity falls through the evening); the magnitudes are design values chosen so the resulting multipliers stay in a "visible but not extreme" band. A sensitivity analysis (reported below) shows the optimal policy is stable to ±15 % perturbation of all of them.

The problem is finite-horizon by construction: T = 12, the episode ends at t = 12, the terminal value is operating_value(s), and the solvers use backward induction. The demand/supply balance state is assumed observed, so the model is a fully observed MDP, not a POMDP.

State-transition diagram — Case Study 1

The diagram is bipartite: the rectangular nodes are the nine states (the seven-level demand/supply balance axis SEVERE_OVERSUPPLY … BALANCED … SEVERE_UNDERSUPPLY, plus CRITICAL_BACKLOG and the structural shortage state LOW_DRIVER_BASE), and the rounded nodes are the five actions. An edge from a state to an action means that action is available in that state; an edge from an action back to a state is a possible next state, labelled with P(s' | s, a). To keep the figure readable only the higher-probability edges are labelled and edge colours only help separate overlapping arrows. A state → action edge additionally carries R(s, a) = …, the reward for that action in that state; it is a function of (s, a) only and does not depend on the next state. The complete transition table is BASE_SPEC in the notebook (Section 1, transition-model cell).



Case Study 2 — Energy Management in a Smart Grid

Horizon. Hourly decisions over one day give T = 24 decision periods (t = 0 … 23); the day ends at t = 24. The finite horizon is a property of the problem and the solver (backward induction with a terminal value), not of the state — the state carries the current grid condition, and the solver tracks how many hours remain.

States (S = 10)

The state is factored into the two quantities the operator observes: the energy situation — SURPLUS (0), BALANCED (1), DEFICIT (2) — and the battery level — LOW (0), MED (1), HIGH (2). That gives 3 × 3 = 9 normal states, written SITUATION|BATTERY, plus one absorbing-risk state CRITICAL, reached when a deficit is left uncovered — 10 states in total.

Situation	Meaning
SURPLUS	Generation exceeds demand — spare energy to store.
BALANCED	Generation ≈ demand — the ideal operating point.
DEFICIT	Demand exceeds generation — the gap must be closed from storage, imports, or by shedding load.
CRITICAL	A deficit went uncovered — blackout risk; a large penalty applies until the grid recovers.

Battery level is part of the state because it changes which actions are feasible and sensible: an empty battery cannot be discharged, a full one cannot be charged, and a healthy reserve is what lets the grid ride out the evening peak.

Actions (A = 6)

#	Action	Role
0	DISTRIBUTE	Send generated energy straight to the city. The default when generation ≈ demand.

#	Action	Role
1	CHARGE	Store spare generation in the batteries (level rises slowly — hours to fill). Its value is inter-temporal: charge now, spend later.
2	DISCHARGE	Draw stored energy to serve the city (level falls fast). The zero-marginal-cost way to cover a deficit.
3	LOAD_SHED	Supply only critical loads, curtail the rest. Sacrifices customer service to prevent a blackout — also a price-responsive demand action.
4	HOLD	Keep the current configuration; no active dispatch.
5	IMPORT_GRID	Buy power from the neighbouring grid. Reliably closes a deficit but costs money — far more during the evening system peak (time-of-use tariff).

The first five actions map directly onto the levers in the case description (send / store / draw from storage / supply only critical loads / keep things as they are). IMPORT_GRID is an addition made under the assignment's licence to "use your imagination to complement the case study". The case description frames the problem economically throughout ("good or bad outcomes", the cost of choices), and buying from an interconnector is a standard, real option for a city grid. It is included because it lets the model capture the daily economic cycle the case description emphasises — off-peak power is cheap, evening-peak power is expensive — which in turn produces a genuinely time-dependent optimal policy. Its cost and the tariff are stated explicitly below so the addition is transparent.

Rewards and rationale for the reward matrix

$R_t(s, a) = \text{situation_value}(s) + \text{action_effect}(s, a) + \text{tariff_t}(a)$.

situation_value(s): BALANCED = +8, SURPLUS = +4, DEFICIT = −2, CRITICAL = −30. Rationale: BALANCED is the target; SURPLUS is good but slightly less valuable because energy generated and neither used nor stored is partly wasted; a covered DEFICIT is a manageable hour, so its penalty is mild; CRITICAL (a blackout regime) is scored far worse than any single bad hour.

action_effect(s, a) folds each action's operating cost and service outcome into one state-dependent figure. Representative cases: charging while in DEFICIT is penalised because it starves the city to fill a battery; deep-discharging a LOW battery is penalised because it risks the reserve; discharging into a DEFICIT is rewarded because that is what storage is for; LOAD_SHED carries a service penalty, worse still if there is no shortage to justify it. The full matrix is printed in the notebook at MIDDAY and at EVENING.

tariff_t(a) applies only to IMPORT_GRID and is the only time-dependent term in the reward:

Phase	NIGHT	MORNING	MIDDAY	EVENING
import tariff	−2.5	−4.0	−3.0	−11.0

The severe evening tariff is what makes DISCHARGE — and, once the battery is empty, even LOAD_SHED — economically preferable to importing during the peak.

The terminal reward is $V_T(s) = \text{situation_value}(s) + 1.5 \cdot \text{battery_level}$: a calm grid handed over with a charged battery is a good starting point for the next day; a CRITICAL hand-over is not.

Transition probabilities and rationale for the transition matrix

Base model P_base. As in Case Study 1, the distribution over next states is specified explicitly for every (state, action) pair — 60 rows in total — and each row is renormalised to 1. Three principles fix the numbers:

- Situation dynamics — the situation mean-reverts on SURPLUS ↔ BALANCED ↔ DEFICIT: from SURPLUS or DEFICIT it is about as likely to move toward BALANCED as to persist (≈ 0.45 each), and BALANCED mostly stays put (≈ 0.50). LOAD_SHED and IMPORT_GRID pull the distribution away from DEFICIT because they cut demand / add supply.

- Battery dynamics — the battery level moves with the action: CHARGE raises it by one, slowly and only with a real surplus; DISCHARGE lowers it by one fast; IMPORT_GRID can trickle-charge; every other action is a slow drain. The LOW and HIGH bounds are respected.
- Critical-deficit risk — a DEFICIT left uncovered (not DISCHARGE with charge, not IMPORT_GRID, not LOAD_SHED) carries a blackout risk to CRITICAL that is larger at low battery (≈ 0.55 / 0.20 / 0.05 at LOW / MED / HIGH), so a healthy battery passively buffers the grid even in an hour it is not spent. CRITICAL is escaped fastest by LOAD_SHED (≈ 0.8) and IMPORT_GRID (≈ 0.7), more slowly by DISCHARGE (≈ 0.6), and barely at all otherwise (≈ 0.1).

Magnitudes are round (multiples of 0.05); as with Case Study 1, only the ordering and the rough ratios carry meaning, and the sensitivity analysis below perturbs all of them.

Time variation. The 24-hour day is four phases; each applies a supply_bias that tilts the next-state situation toward SURPLUS (positive) or DEFICIT (negative) via exp(0.42 · supply_bias · situation_score[s']):

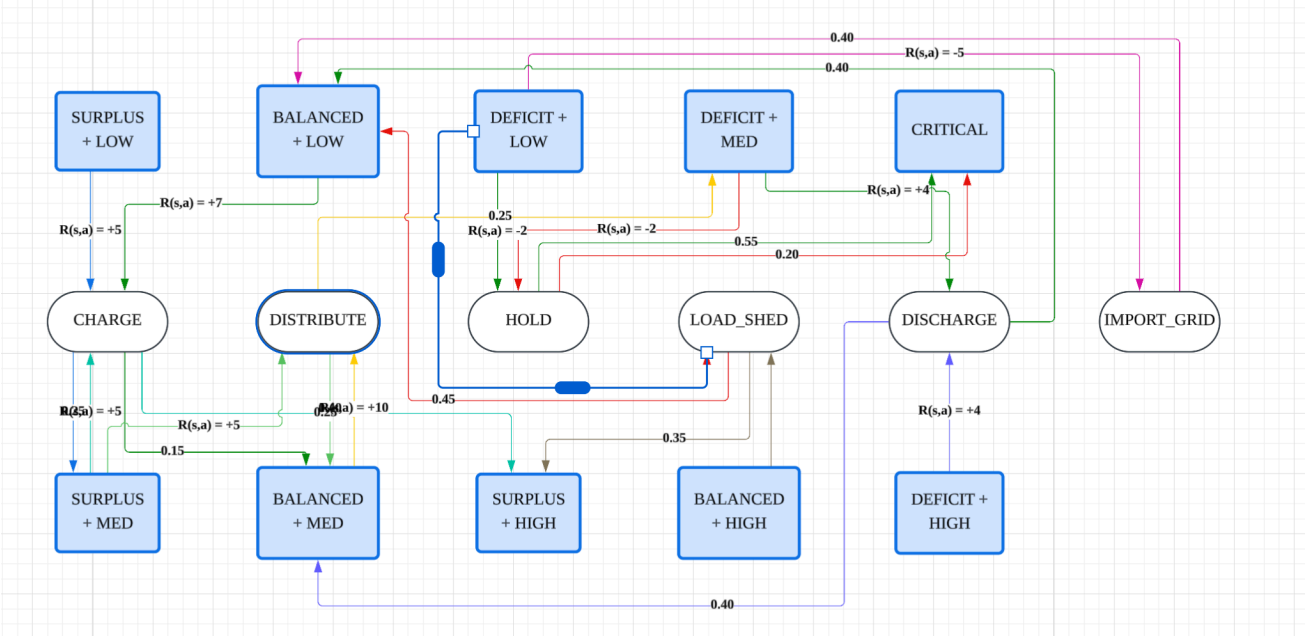
Phase	hours	supply_bias	meaning
NIGHT	0–5	−0.55	no solar, low demand, wind only
MORNING	6–11	+0.35	solar ramps up, demand rises
MIDDAY	12–17	+1.90	solar peak greatly exceeds demand — strong surplus
EVENING	18–23	−2.30	solar gone, demand peak — the hard part of the day

The shape of the cycle is fixed by the physical daily story the case description tells (surplus at midday, deficit in the evening); the magnitudes are design values, and the sensitivity analysis below shows the policy is stable to ±15 % perturbation.

The problem is finite-horizon: T = 24, the day ends at t = 24, the terminal value rewards a calm, charged hand-over, and the solvers use backward induction. The state (situation, battery, and the CRITICAL flag) is assumed observed each hour, so the model is a fully observed MDP, not a POMDP.

State-transition diagram — Case Study 2

As with Case Study 1, the diagram is bipartite: rectangular nodes are the ten states (the nine situation × battery combinations — situation DEFICIT / BALANCED / SURPLUS, battery LOW / MED / HIGH — plus CRITICAL) and rounded nodes are the six actions. A state → action edge marks an available action; an action → state edge is a possible outcome labelled with P(s' | s, a); only the higher-probability edges are labelled and edge colours only help separate overlapping arrows. Each state → action edge also carries R(s, a) = …, the reward for that action in that state (a function of (s, a) only; for IMPORT_GRID the value shown is the midday tariff — see the reward section for the time-of-use variation). Reading the actions: CHARGE raises the battery level, DISCHARGE spends it to cover a deficit, LOAD_SHED and IMPORT_GRID lift the situation and pull the state out of CRITICAL. The full transition table is BASE_SPEC in the notebook (Section 1, transition-model cell).



Experimental results and discussion

a. Experimental settings

Setting	Case Study 1	Case Study 2
States S	9	10
Actions A	5	6
Horizon T	12 (10-minute periods)	24 (hourly)
Discount γ	1.0 (finite horizon)	1.0 (finite horizon)
Dynamics	time-varying P_t, 4 phases	time-varying P_t and R_t, 4 phases
Value iteration stopping rule	exact after T backward stage-backups	exact after T backward stage-backups
Policy iteration initial policy	HOLD in every (t, s)	DISTRIBUTE in every (t, s)
Policy iteration stopping rule	policy unchanged over a full improvement pass	policy unchanged over a full improvement pass
Convergence timing	best of 200 repeated solves (time.perf_counter)	best of 200 repeated solves
Simulation	20,000 Monte-Carlo episodes from a fixed start distribution	20,000 Monte-Carlo episodes

All randomness is seeded, so the notebooks reproduce these figures exactly; only the millisecond timings are machine-dependent.

b. Experimental results

i. Performance

For both case studies, value iteration and policy iteration return the identical optimal non-stationary policy and the identical optimal value function (checked with np.array_equal and np.allclose), and their simulated average rewards are identical up to Monte-Carlo noise. The DP solution is also clearly worth having:

Quantity	Case Study 1	Case Study 2
Optimal-policy average reward (20k episodes)	64.56 (σ ≈ 24.4)	136.39 (σ ≈ 30.5)
Naïve-baseline average reward	17.82 (always HOLD)	−160.77 (always DISTRIBUTE)
Greedy (immediate-reward) policy vs optimal	differs on 7 of 9 states	differs on 3 of 10 states

The baselines are poor for instructive reasons: always-HOLD lets a ride-sharing shortage tip into

CRITICAL_BACKLOG, and always-DISTRIBUTE walks the grid into CRITICAL on the first evening deficit that meets an empty battery (hence its large negative mean and huge variance). A purely greedy policy — pick the highest immediate reward — disagrees with the optimum on most Case Study 1 states, confirming that the value of the DP solution comes from accounting for where each action leads, not just its immediate cost.

Convergence. In a finite horizon the two algorithms differ in how they reach the optimum. Value iteration performs exactly T backward stage-backups — 12 for Case Study 1, 24 for Case Study 2 — each one a cheap max over actions, and is exact after that single sweep. Policy iteration converges in 4 improvement passes for Case Study 1 and 5 for Case Study 2, but each pass runs a full T-stage policy evaluation followed by a full improvement sweep, so it performs several times more arithmetic in total. The net effect on wall-clock time is that value iteration is faster for both problems (roughly 8× for Case Study 1 and 10× for Case Study 2 on the test machine), even though it "runs more iterations" — a value-iteration iteration and a policy-iteration iteration are different units of work.

Robustness. P_base, the reward numbers and the phase parameters are all hand-designed, so the notebooks re-solve each MDP many times with every such value perturbed. Across 300 perturbed MDPs for Case Study 1 (R ±10 %, P_base ±15 %, phase and tilt parameters ±15 %), the first-period optimal policy is unchanged on 88 % of states on average and the full (t, s) policy grid on 87 %. For Case Study 2 (200 perturbed MDPs, similar perturbations) the figures are 96 % and 96 %. The qualitative policy structure does not depend on the precise parameter values.

Policy interpretation. In Case Study 1 the cause of a shortage flips through the evening: during the peak a shortage is demand-driven and is met with demand-side tools (RAISE_PRICE, or PAUSE_NEW_REQUESTS when severe), whereas late at night the same shortage state is supply-driven — drivers have logged off — so the optimal action switches to the supply-side DRIVER_INCENTIVE, and a mild shortage is simply held because a price change would do nothing with so few riders active. Near the end of the horizon the over-supply states drop LOWER_PRICE for HOLD: with no future periods left, a discount that only pays off later is no longer worth its cost. In Case Study 2 the optimal strategy is battery energy arbitrage: charge cheap solar energy through the day, ration the battery in the run-up to the evening peak (small deficits covered by still-cheap imports), then discharge hard during the peak, and once the battery is empty switch from IMPORT_GRID to LOAD_SHED because at the −11 peak tariff shedding non-critical load is genuinely cheaper. In the final hour the SURPLUS states stop charging — energy stored in the last hour can never be used — a clean example of the turnpike property (the policy is stationary in the interior of the horizon and deviates only near the endpoints).

ii. Comparative study

Case Study	Algorithm	Optimal Policy	Optimal Value Function (mean of V(0,·))	Convergence Time	Number of Iterations	Average Reward
1	Policy Iteration	identical to VI	62.15	≈ 0.16 ms	4	64.56
1	Value Iteration	identical to PI	62.15	≈ 0.02 ms	12	64.56
2	Policy Iteration	identical to VI	132.29	≈ 0.40 ms	5	136.39
2	Value Iteration	identical to PI	132.29	≈ 0.04 ms	24	136.39

First-period optimal policy $\pi^*(0, \cdot)$:

- Case Study 1: BALANCED → HOLD; MILD_UNDERSUPPLY and MODERATE_UNDERSUPPLY → RAISE_PRICE; SEVERE_UNDERSUPPLY → PAUSE_NEW_REQUESTS; MILD_OVERSUPPLY → HOLD; MODERATE_OVERSUPPLY and SEVERE_OVERSUPPLY → LOWER_PRICE; LOW_DRIVER_BASE → DRIVER_INCENTIVE; CRITICAL_BACKLOG → PAUSE_NEW_REQUESTS.
- Case Study 2: the SURPLUS states CHARGE (or DISTRIBUTE when the battery is already HIGH); the BALANCED states DISTRIBUTE; DEFICIT|LOW → IMPORT_GRID; DEFICIT|MED and DEFICIT|HIGH → DISCHARGE; CRITICAL → LOAD_SHED.

Implications:

- Solution quality is identical. The two algorithms are alternative routes to the same Bellman-optimal fixed point, so this is expected, and the fact that two independent from-scratch implementations agree exactly on two different problems is a strong correctness signal.
- The cost trade-off is the classic one, re-derived in the finite-horizon setting. Value iteration spreads its work over many light stage-backups; policy iteration concentrates its work into a few heavy passes, each solving the policy exactly over the whole horizon. Policy iteration converges in far fewer iterations, but for problems of this size the

per-iteration cost dominates and value iteration wins on wall-clock time. Policy iteration's advantage would show on a problem where evaluating a fixed policy is cheap relative to the number of value-iteration sweeps needed — for example a long-horizon discounted problem where value iteration needs hundreds of sweeps to converge.

- Both policies are strongly non-stationary: 7 of the 9 Case Study 1 states and 6 of the 10 Case Study 2 states change their optimal action over the episode.

iii. Discussion

Value iteration — strengths. In a finite horizon it is the natural and minimal algorithm: one backward sweep, exact after T stage-backups, no convergence tolerance to choose, trivial to implement and vectorise, and the faster of the two in wall-clock time on both case studies.

Value iteration — limitations. The iteration count equals the horizon and cannot be reduced. On a long-horizon discounted problem the analogous algorithm converges only geometrically (about γ^k per sweep) and can need many iterations for a tight tolerance, and it computes the full value function even when only the greedy policy is wanted.

Policy iteration — strengths. It converges in very few outer iterations (4 for Case Study 1, 5 for Case Study 2) because each policy evaluation is exact; the intermediate policies are themselves usable; and the iteration count is insensitive to the horizon length. On problems where policy evaluation is cheap relative to the number of value-iteration sweeps required, it is the faster method.

Policy iteration — limitations. Each iteration is expensive — a full-horizon evaluation plus an improvement sweep — so on small problems like these it does several times more total arithmetic than value iteration and is slower in wall-clock time. A correct implementation must also ensure the policy evaluation is solved consistently, here exactly by backward recursion.

Which method suits these problems. Both case studies are small ($S \leq 10$, $T \leq 24$) and undiscounted, which is exactly the regime where value iteration's simplicity wins: it is faster, has no free parameters, and its single backward sweep matches the structure of the problem. Policy iteration is valuable here mainly as an independent check — a completely different algorithm reproducing the identical policy and value function on both problems is strong evidence that the formulations and the implementations are correct. On a larger or discounted version of either problem — a multi-zone ride-sharing network, or a grid problem solved over a rolling multi-day horizon — the balance would shift and policy iteration's small iteration count would begin to matter.

Conclusion

Two real-world control problems were formulated as finite-horizon MDPs — a 12-period dynamic-pricing problem for a ride-sharing zone and a 24-hour energy-management problem for a smart grid — each with 9–10 fully observed states, 5–6 actions, hand-designed transition and reward models with stated rationale, and time-varying dynamics that make the finite horizon meaningful. Both were solved from scratch with value iteration (backward induction) and policy iteration.

The two algorithms returned the identical optimal non-stationary policy and value function on both problems, and both policies substantially outperformed naïve and greedy baselines (64.56 versus 17.82 for Case Study 1; 136.39 versus −160.77 for Case Study 2), confirming that the value of dynamic programming lies in anticipating where actions lead. Value iteration was the faster method for both problems because each of its T stage-backups is cheap, whereas policy iteration's few iterations are each a full-horizon evaluation; policy iteration's role here is as an exact, independent verification of the solution. The resulting policies are interpretable and match intuition — a graded, time-of-day-dependent response to demand/supply imbalance in the ride-sharing zone, and battery energy arbitrage in the grid — and a sensitivity analysis showed this structure is robust to ±10–15 % perturbation of the hand-chosen parameters. The main lesson is that a careful finite-horizon MDP formulation, solved exactly by dynamic programming, turns an informal operational description into a precise, non-stationary decision rule whose behaviour can be explained period by period.